SL2DT-09: Cutover, Validation & Decommission

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 9 of 11 | **Created:** April 2026 | **Last Updated:** 10/05/2026

Overview

Goal of this step: run the parallel validation window, declare cutover, and decommission Sumo. This is the most operationally risky phase — a bad cutover means on-call teams missing alerts or dashboards. Discipline matters.

The work here is mostly procedural: follow the runbook, execute gates, document outcomes. The engineering was done in SL2DT-03 through SL2DT-08.

Table of Contents

1. What You'll Produce
2. Three-Tier Validation Model
3. Parallel-Run Window — Setup & Discipline
4. Validation Gates — Per Wave
5. The Cutover Runbook
6. Rollback Plan
7. Sumo Decommission
8. Post-Cutover First 30 Days
9. Final Step Exit Criteria
10. References

Prerequisites

Requirement	Details
Audience	Migration lead, SRE, on-call teams, change management
Inputs	Everything from SL2DT-01 through SL2DT-08
Dynatrace access	Full write on all tenants; on-call access for alerts
Sumo access	Admin (needed for decommission + data export)
Prior reading	All preceding SL2DT notebooks

1. What You'll Produce

Artifact	Purpose
`cutover/validation-report.md`	Three-tier validation results
`cutover/runbook.md`	Step-by-step cutover commands + checkpoints
`cutover/rollback-plan.md`	What to do if cutover goes wrong
`cutover/decommission-checklist.md`	Sumo teardown steps
`cutover/post-cutover-retro.md`	Retro after 30-day stable period

2. Three-Tier Validation Model

Don't assume "looks fine in Dynatrace." Validate formally across three tiers.

Tier 1 — Technical Parity (automated)

Mechanical comparison between Sumo and Dynatrace for the same source data.

Check	Method	Pass Criteria
Ingest volume	Bucket count/hr vs Sumo `_sourceCategory` count/hr	±5% for all scopes

Check	Method	Pass Criteria
Query results	Sample 100 DQL queries against Sumo equivalents	≥90% match
Monitor firing	Force-trigger condition in both tools	Both fire within 2 min of each other
Alert routing	Trigger → verify notification delivery	100% delivery in DT

Tier 2 — Functional Equivalence (semi-automated + human review)

Business-level features work as expected.

Check	Method	Pass Criteria
Dashboard panel fidelity	App-team spot-checks	≥90% acceptance per team
Alert noise	Week-over-week alert count vs Sumo baseline	≤ Sumo baseline
Investigation time	Try to root-cause 5 past incidents using DT	Comparable or faster than Sumo
User access	Each user logs in, sees expected dashboards/data	100% users confirmed

Tier 3 — Business Signoff (human)

Each team and exec sponsor formally signs off.

Stakeholder	Signoff
Each app team lead	✅ / ❌ per team
Platform engineering lead	✅ / ❌
Security / compliance	✅ / ❌ (audit trail parity)
Exec sponsor	✅ / ❌

3. Parallel-Run Window — Setup & Discipline

Setup

Both Sumo and Dynatrace receive the same data. Duration: **2–4 weeks** typical. Longer is a warning sign — probably hiding a data gap.

Dual-Ingest Patterns

For each source, choose a dual-ingest mechanism:

Source	Dual-Ingest Strategy
Syslog	Forward to both Sumo collector + OTel Collector
Kubernetes	DynaKube logs + Sumo FluentBit (in parallel)
AWS CloudWatch	AWS → Sumo + AWS Clouds app
HTTP source	Teams change nothing; intermediate proxy forks to both

Discipline Rules

1. **Freeze Sumo changes.** No new dashboards, monitors, FERs in Sumo during parallel run. Everything happens in Dynatrace.
2. **Monitor alert parity daily.** Track: same alert fires in both, alert fires in one only, new DT alert.
3. **Daily standup review.** Migration lead + SRE rep + on-call rep. Surface divergences.
4. **Parallel-run exit criteria explicit.** Define before starting. Common criteria:
   - Tier 1 passes 7 consecutive days
   - Tier 2 team signoff ≥ 80% of teams
   - Alert volume in DT ≤ Sumo baseline

Validation Queries

```
// Parallel-run parity check: DT log volume per scope
fetch logs, from:-24h
| summarize c = count(), by:{sumo.source_category}
| sort c desc
| limit 100
```

```
// Parallel-run alert volume (DT detected problems, one record per problem)
fetch dt.davis.problems, from:-24h
| summarize problems = count(), by:{event.category}
| sort problems desc
```

4. Validation Gates — Per Wave

Each migration wave (SL2DT-01 §3) had entry/exit criteria. Validate them cumulatively:

Wave	Validate
Wave 0 — Cut scope	Retired assets are actually unused (query audit log)
Wave 1 — Ingest parity	All buckets receiving data ±5% of Sumo volume
Wave 2 — Core monitors	Top-10 monitors confirmed firing correctly
Wave 3 — Dashboards	In-scope dashboards live + app-team signoff
Wave 4 — Remaining monitors/dashboards	100% of migrate-list complete
Wave 5 — Cutover	See below

Daily Validation Checklist (during parallel run)

- [ ] All expected buckets receiving data (no zero-volume scopes)
- [ ] detected problem count within range (not an explosion)
- [ ] No integration failures (ServiceNow delivery, Slack, PagerDuty)
- [ ] No user access tickets

- [ ] No data-loss escalations

Source-level ingest coverage (Wave 1 parity)

> **Coverage vs. volume — the log-module self-monitoring events.** For Wave 1 ingest parity, verify coverage at the *source* level, not just bucket volume — a source detected but not ingesting is a silent parity gap that volume checks miss. The log module emits a `log_source.status` event per discovered source into `dt.system.events`, carrying `log.source.file_status` and `log.source.ingest_status` — so it reveals sources OneAgent **detected but is not ingesting**, which a `fetch logs` count cannot show (no stored records means nothing to count). It scans events, not raw logs, and needs only event-read.
>
> This event stream is **Early Access** and requires **opt-in** (the OneAgent log module must be enabled to send self-monitoring events; its content folds into the built-in *Log ingest Overview* dashboard for Dynatrace **1.339+** — verify it is flowing in your tenant before relying on it). See **FAQ-08** (Recommended Approach) for the full file-status × ingest-status coverage matrix and the `fetch logs` fallback for tenants without the opt-in.

```
// Log-source coverage from the OneAgent log-module self-monitoring events.
// Unlike `fetch logs` (which sees only sources that produced records), this
// surfaces sources OneAgent DETECTED but is NOT ingesting. Scans events, not
raw logs.
fetch dt.system.events, from: now() - 24h
| filter event.provider == "Log Module" and event.type == "log_source.status"
| dedup event.id, sort:{ timestamp desc }
| filter event.status == "Active"
| fieldsAdd coverage =
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Ingested", "Available | Ingesting", else:
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Not ingested", "Available | NOT ingesting", else:
    if(log.source.ingest_status == "Partially ingested", "Partial ingestion",
    else: "Unsupported or issue")))
| summarize sources = count(), by:{coverage}
| sort sources desc
```

5. The Cutover Runbook

Cutover = moment when Sumo is no longer authoritative. On-call pages route only from Dynatrace. Dashboards displayed on war-room screens are DT.

T-14 Days: Pre-Cutover Prep

- [ ] Final parallel-run validation report published
- [ ] All team signoffs collected
- [ ] Exec sponsor formal go/no-go
- [ ] Change freeze announced for Sumo + Dynatrace (except migration team)
- [ ] On-call schedule locked + escalation paths tested
- [ ] Rollback plan reviewed by SRE lead

T-3 Days: Final Validation

- [ ] Repeat Tier 1 check — green
- [ ] All ServiceNow/Slack integrations tested with live incident
- [ ] User access spot-check (5 users per team)

T-0: Cutover Day

```
HH:00  Cutover announcement (all-hands)
HH:00  Freeze all Sumo + DT config changes
HH:30  Stop Sumo-side alerting (pause all monitors)
HH:45  Verify DT alerts still firing correctly
HH:60  Monitor for 60 min — escalation criteria defined below
HH:120 Proceed with decommission, or roll back per criteria
```

Escalation Criteria During Cutover Window

Signal	Action
DT missing a critical alert that Sumo caught	Re-enable Sumo alerts, investigate, re-cut later
ServiceNow delivery failures	Re-enable Sumo alerts for affected routes, investigate

Signal	Action
Major dashboard broken	Post workaround; keep cut-over if non-critical
Ingest gap > 10 min	Investigate immediately; may be cutover-related

Post-Cutover (T+1 Day)

- [ ] 24h of alerting without Sumo as fallback
- [ ] No escalated incidents attributable to migration
- [ ] On-call rotation handoff completed cleanly

6. Rollback Plan

Rollback is: re-enable Sumo alerting while keeping DT ingest running. This is a one-way door only at Sumo contract termination — until then, roll back freely if needed.

Rollback Triggers

Trigger	Action
DT ingest pipeline breaks	Re-enable Sumo ingest + alerts; investigate
Critical alert silent for > 30 min	Re-enable Sumo monitor for that alert class
Cross-system integration failure (ServiceNow etc.)	Re-enable Sumo until fix deployed
Data loss detected	Full rollback; pause cutover

Rollback Runbook

```
1. Unpause Sumo monitors (group-level enable)
2. Verify Sumo ingest hasn't fully stopped (should still be running per
parallel-run setup)
3. Route on-call notifications back to Sumo-sourced alerts
4. Announce rollback; update incident channel
5. Investigate root cause; do not attempt re-cut until resolved
```

Post-Rollback Steps

- Run full validation Tier 1 + Tier 2 again
- Address the root cause
- Schedule new cutover date with stakeholder approval

Anti-Patterns

- **Partial rollback** (some teams on Sumo, some on DT). Confusing; don't.
- **Rolling back Dynatrace config.** Keep DT state — rolling back IAM or bucket config is a larger undertaking.
- **"Let's just watch it" when alerts go silent.** Silence isn't success. Re-enable Sumo.

7. Sumo Decommission

Only after T+30 days of stable DT operation.

Pre-Decommission Checks

- [ ] All teams signed off (Tier 3)
- [ ] No rollbacks in last 30 days
- [ ] DT operating at full volume
- [ ] Compliance retention confirmed for both Dynatrace audit records — environment audit events (`dt.system.events`, one year) and the Account Management audit log — with an export planned where longer retention is required (SL2DT-07 §6)
- [ ] Sumo data archival complete (per retention policy — regulated industries may require long-term archive)

Decommission Sequence

1. **Export inventory** (final backup) — dashboards, monitors, FERs, settings, using the archival script below
2. **Archive critical data** — if required: export to S3/GCS with compliance retention; verify the archive opens
3. **Cancel Sumo ingest sources** — disable Collectors; stop dual-writing
4. **Disable SSO integration** — Sumo stops accepting new logins

5. **Revoke Sumo API keys** — last, because steps 1–2 use them; confirm no downstream automation still reads Sumo
6. **Wait for confirmation** — 14-day window with no access requests
7. **Terminate Sumo contract** — per contractual notice period

Data Archival

```
# Export all dashboards (final snapshot) — run before the API keys are revoked
# SUMO_API as set in SL2DT-02 §2 (us1 has no region label in its host)
mkdir -p archive/dashboards
for id in $(jq -r '.dashboards[].id' final-inventory/dashboards.json); do
  curl -u "$SUMO_ACCESS_ID:$SUMO_ACCESS_KEY" \
    "$SUMO_API/api/v2/dashboards/$id" &gt; archive/dashboards/$id.json
done
tar czf sumo-final-archive.tar.gz archive/
# Upload to long-term storage
aws s3 cp sumo-final-archive.tar.gz s3://compliance-archive/sumo/
```

8. Post-Cutover First 30 Days

Week 1 — Stabilization

- Daily war-room calls (migration lead + SRE + on-call)
- Daily alert volume review
- Hot-patch any remaining low-confidence translations that fire false positives
- User access ticket triage (expect a burst)

Week 2 — Tuning

- anomaly detection baselines stabilize — evaluate detector thresholds
- Reduce alert-fatigue items (obvious duplicates, too-narrow thresholds)
- App-team followups on dashboard polish

Week 3 — Handoff

- On-call fully owns DT (no migration-team escalation)
- Migration team transitions to advisory role

- Lessons learned doc draft

Week 4 — Retro

- Formal retro with all stakeholders
- What went well, what didn't, what would we change
- Document in `cutover/post-cutover-retro.md`
- Feed back into shared skill (sumoql-to-dql) if translation gaps were discovered

Metrics to Track

Metric	Target	Actual
Alert noise (DT alert count / week)	≤ Sumo baseline	_____
User access tickets	< 10 / week	_____
Dynatrace Intelligence adoption rate	≥ 40% of new monitors	_____
Translation low-confidence rewrites needed	≤ 30 total	_____
MTTR post-cutover	≤ pre-cutover	_____

9. Final Step Exit Criteria

G9 — Migration Complete

- [ ] All three validation tiers passed
- [ ] Cutover executed per runbook, no rollback triggered
- [ ] 30-day post-cutover stability achieved
- [ ] All teams signed off (Tier 3)
- [ ] Sumo decommissioned + contract terminated
- [ ] Retro complete; lessons learned documented
- [ ] `sumoql-to-dql` skill updated with any new translation patterns discovered

You're done. Hand off to steady-state operations. Start scoping the next project (phase 2 tool migrations, if any).

Next: SL2DT-99 — Summary & Runbook Index (full runbook compendium + quick-start for future migrations).

10. References

Dynatrace migration validation surfaces

- OpenPipeline (DT docs)
- DQL Reference (DT docs)
- Anomaly detection (DT docs)
- Problems app (DT docs)
- Maintenance windows (DT docs)

Sumo Logic decommission references (source)

- Sumo Logic API (Sumo Logic docs)
- Sumo Logic users and roles (Sumo Logic docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.